

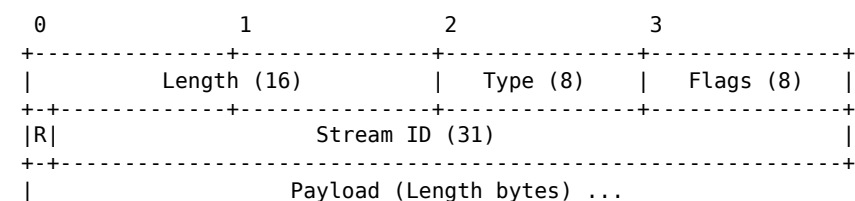
BH/1: HTTP in Binary Frames

BH/1 carries HTTP requests and responses (methods, paths, status codes, headers, bodies) over one TCP connection. Instead of text lines, everything travels as length-prefixed binary frames. This spec is everything a stranger needs to build a server or a client. MUST, SHOULD and MAY mean what they mean in RFC 2119. All integers are big-endian (network byte order). All strings are UTF-8.

1. Connection

1. The client opens **one** TCP connection and sends the 4-byte **preface** 42 48 2F 31 (ASCII BH/1). It sends the preface once, before anything else.
2. After the preface, both sides send only frames (§2).
3. The connection stays open for any number of requests. The client closes it when it is done. The server **MUST NOT** close it after a response. It closes only after a bad preface (§6).
4. The server answers requests in the order it receives them. A client **MUST NOT** open a second connection to the same server.

2. Frame header (8 bytes, fixed)



Field	Bits	Meaning
Length	16	Number of payload bytes after the header, 0 to 65 535. The header itself is not counted.
Type	8	0x00 DATA, 0x01 HEADERS. All other values are unknown (§5).
Flags	8	0x01 END_STREAM: the last frame of this request or response. Other bits MUST be sent as 0 and ignored on receipt.
R	1	Reserved. MUST be sent as 0 and ignored on receipt.
Stream ID	31	Which request/response the frame belongs to. The client numbers requests 1, 2, 3... The response uses the same ID. ID 0 means "the connection itself".

3. Header block (the payload of a HEADERS frame)

A header block is a list of fields placed back to back, with no count and no terminator. The frame's Length says where it ends. Each field is:

```

Indexed name: 1xxxxxxx | ValueLen (16) | Value
Literal name: 00000000 | NameLen (8) | Name | ValueLen (16) | Value

```

In an **indexed** field the first byte's high bit is 1, and the low 7 bits give an index from 1 to 10 in the static table below. In a **literal** field the first byte is 0x00, followed by the name (1 to 255 bytes, lowercase ASCII). Any other first byte, or an index outside 1-10, is malformed. Names starting with : are pseudo-headers.

#	Name	Sent by	#	Name	Sent by
1	:method	client	6	accept	client
2	:path	client	7	content-type	server
3	:status	server	8	content-length	server
4	host	client	9	server	server
5	user-agent	client	10	date	server

The table is the ten names our programs actually send, so a normal exchange never spells a name out (HPACK's static table). Any other name still works as a literal (HPACK's length-prefixed string). A receiver **MUST** ignore literal names it does not understand.

4. Messages

Request. A request is exactly one HEADERS frame with END_STREAM set and Stream ID > 0. It MUST contain :method and :path, and :path MUST start with /. Version 1 defines only GET. Requests have no body.

Response. The server sends one HEADERS frame on the request's Stream ID. :status (three ASCII digits) is its first field. If the body is empty, that HEADERS frame carries END_STREAM. Otherwise zero or more DATA frames follow, each carrying the next part of the body, and the last one carries END_STREAM. content-length gives the total body size. A sender MAY split a body into DATA frames of any size up to 65 535 bytes.

Status codes.

- 200: the file's bytes.
- 404: no such file under the root. / maps to /index.html. A path that escapes the root, such as ../../, also gets 404.
- 405: a method other than GET.
- 400: malformed request (§6).

5. Unknown frame types: MUST skip

A receiver that meets a frame whose Type it does not know **MUST read and discard exactly Length payload bytes and carry on** with the next frame. It sends no error and does not close the connection. This is always possible, because Length sits at the same place in every frame, whatever the type. This rule is what makes a version 2 possible: a v2 peer can send new frame types (for example PING or PRIORITY), and a v1 peer passes over them untouched.

6. Errors

A request is **malformed** if any of the following hold:

- its header block cannot be decoded (a value runs past the end, a bad first byte, an index outside 1–10);
- :method or :path is missing, or :path does not start with /;
- it arrives on Stream ID 0;
- its HEADERS frame lacks END_STREAM;
- a client sends a DATA frame.

The server answers **400 on the same Stream ID and keeps the connection open**. The frame header was valid, so the server still knows where the next frame starts.

A **bad preface** is different. Nothing after it can be trusted to line up with frame boundaries. The server sends a 400 response on Stream ID 0, then closes the connection.

7. Why these widths (16 / 8 / 8 / 1+31)

HTTP/2 uses a 9-byte header: 24 / 8 / 8 / 1+31. It needs 24 bits of length because frame size is *negotiable*: SETTINGS_MAX_FRAME_SIZE can raise it from 16 KiB up to 16 MiB. It needs 31 bits of stream ID because many streams share one connection and IDs are never reused. Our choices:

- **Length 16 bits.** BH/1 has no SETTINGS frame, so the field width *is* the limit, and every receiver knows the limit without asking. 64 KiB bounds the memory a receiver needs for one frame. A file of any size is just more DATA frames: at 16 KiB per frame the header costs 8 bytes per 16 384, about 0.05%. It also makes the header 8 bytes, exactly two 32-bit words, instead of HTTP/2's odd 9.
- **Type 8 bits.** Version 1 uses 2 of 256 values. That leaves 254 for later versions, and §5 makes adding them safe.
- **Flags 8 bits.** One flag is used, seven are spare, and unknown flag bits are ignored.
- **R + Stream ID 31 bits.** Version 1 handles one request at a time, so this could look unnecessary. It is kept for two reasons. First, every response names the request it answers, so a client can check it got the right one. Second, v2 can add multiplexing without changing the header. It is 31 bits, not 16, because the connection is meant to last: a 16-bit ID would run out after 65 535 requests and force the second connection the brief forbids. The reserved bit copies HTTP/2 and gives one spare bit in case a later version needs a signal.
- **Preface BH/1.** Without a preface, a plain-HTTP client's GET / would be read as Length 0x4745 (18 245), and the server would wait forever for bytes that never come. The preface catches that immediately and also carries the version number.